Code Explanation

Batch Processing Test Code Explanation

The provided code is a test suite written in Pytest for testing batch processing functions using PySpark. It includes tests for cleaning customer and order data, as well as verifying the correctness of the cleaning process.

Overview

The test code is designed to validate the functionality of batch processing functions, specifically data cleaning and aggregation. It uses PySpark to create sample data and test the functions against expected outcomes.

Step 1: Creating a SparkSession Fixture
This step involves creating a PySpark SparkSession fixture that can be used throughout the test suite. The fixture is used to create a SparkSession with a specific configuration.

```
@pytest.fixture
def spark():
    """
    Create a SparkSession for testing
    """
    return SparkSession.builder
        .appName("TestCustomerOrderProcessing")
        .master("local[1]")
        .getOrCreate()
```

Step 2: Creating Sample Customer Data Fixture
In this step, sample customer data is created using a PySpark DataFrame. The data includes various scenarios such as null values and duplicate records.

```
@pytest.fixture
def sample_customer_data(spark):
    """
    Create sample customer data for testing
    """
    schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", "Alice Brown", "alice@example.com", "West"),
        ("C005", None, "invalid@example.com", "North"),
        ("C001", "John Doe", "john@example.com", "North")  # Duplica
te
    ]
```

```
    return spark.createDataFrame(data, schema)
```

Step 3: Creating Sample Order Data Fixture
Similar to Step 2, this step creates sample order data with various scenarios.

```
@pytest.fixture
def sample_order_data(spark):
    """
    Create sample order data for testing
    """
    schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", StringType(), True),
        StructField("CustId", StringType(), True)
    ])

    data = [
        ("O001", "Product A", 10.0, 2, "2023-01-15", "C001"),
        ("O002", "Product B", 15.0, 1, "2023-01-20", "C002"),
        ("O003", "Product C", 20.0, 3, "2023-01-25", "C003"),
        ("O004", "Product D", 25.0, 1, "2023-01-30", "C004"),
        ("O005", "Product E", 30.0, None, "2023-02-05", "C001"),
        ("O001", "Product A", 10.0, 2, "2023-01-15", "C001")  # Dupl
icate
    ]

    return spark.createDataFrame(data, schema)
```

Step 4: Testing Customer Data Cleaning
This step tests the `clean_customer_data` function by verifying that it removes null values and duplicates.

```
def test_clean_customer_data(spark, sample_customer_data):
    """
    Test cleaning customer data
    """
    cleaned_df = clean_customer_data(sample_customer_data)

    # Check if nulls are removed
    assert cleaned_df.filter("Name IS NULL").count() == 0

    # Check if duplicates are removed
    assert cleaned_df.count() == 4

    # Check if all required columns exist
    required_columns = ["CustId", "Name", "EmailId", "Region"]
    assert all(col in cleaned_df.columns for col in required_columns
)
```

Step 5: Testing Order Data Cleaning and TotalAmount Calculation
This step tests the `clean_order_data` function by verifying that it removes null values, duplicates, and correctly calculates the `TotalAmount` column.

```
def test_clean_order_data(spark, sample_order_data):
    """
    Test cleaning order data and adding TotalAmount column
    """
    cleaned_df = clean_order_data(sample_order_data)

    # Check if nulls are removed
    assert cleaned_df.filter("Qty IS NULL").count() == 0

    # Check if duplicates are removed
    assert cleaned_df.count() == 4

    # Check if TotalAmount column is added
    assert "TotalAmount" in cleaned_df.columns

    # Verify TotalAmount calculation
    row = cleaned_df.filter("OrderId = 'O001'").first()
    assert row["TotalAmount"] == row["PricePerUnit"] * row["Qty"]
```